WHITEPAPER · PROTOCOL SECURITY / EVASION

Exploiting Logic Asymmetry in Modern Web Application Firewalls

ACED & IBTD Attack Vectors

This research demonstrates that even the most modern WAFs remain vulnerable to attacks exploiting logic asymmetry in HTTP protocol processing. Real-world testing on a Weaver Ecology OA system achieved a 100% bypass rate (40/40 test cases), confirming the critical severity of this architectural flaw.

Author Tuan, 16 years old

Release June 6, 2026

Version 2.0

Classification Protocol Security / Evasion

SECURITY RESEARCH

Table of Contents

I. Abstract

Over the past decade, Web Application Firewalls (WAFs) have evolved from simple string-matching filters into AI-driven intrusion detection systems with sophisticated behavioral analysis. However, this research demonstrates that even the most modern WAFs remain vulnerable to attacks exploiting logic asymmetry in HTTP protocol processing. This asymmetry arises from fundamental differences in how perimeter security devices (WAFs) and application servers (Backends) interpret and process the same HTTP data stream.

This paper introduces and analyzes two attack techniques: **Asymmetric Content-Encoding Deception (ACED)** and **Identity-Based Transparency Deception (IBTD)**. These techniques do not exploit software bugs or memory overflows; rather, they attack the inconsistency in data processing workflows between the perimeter security device and the application server. They allow attackers to forward malicious payloads (SQL Injection, RCE, XXE) through hardened security systems without leaving a trace, creating a severe security blind spot that conventional monitoring systems cannot detect.

This research emphasizes that this is not a code bug fixable with a single patch; it is an architectural design flaw demanding a shift in security mindset: from "trusting standards" to "verifying everything." Real-world testing shows bypass success rates of 85-95% across popular commercial WAFs, and 100% on a live Weaver Ecology OA system (40/40 test cases bypassed successfully), demonstrating that this vulnerability is not merely theoretical but a critical real-world threat.

II. Introduction

2.1 Current Landscape

Organizations today rely on WAFs as an essential layer of protection for their web applications. The traditional threat model focuses on blocking malicious signatures or anomalous behaviors. To counter increasingly sophisticated attacks, WAF vendors have continuously upgraded their technology, integrating machine learning, semantic analysis, and real-time behavioral anomaly detection. However, one of the most common performance optimization decisions has created a critical vulnerability.

To optimize performance, WAFs classify data streams: streams requiring decompression (like Gzip/Brotli) are routed to heavy processing tools (Deep Packet Inspection), while plain text streams are processed faster using lighter pipelines. This classification relies on reading the Content-Encoding header to determine the processing pipeline. This is precisely the point an attacker can exploit: by manipulating headers, an attacker can misroute data streams into the wrong processing pipeline, thereby evading security inspection mechanisms.

2.2 Core Problem

The reliance on performance and high availability has created a "deadly gap" in security architecture. This research demonstrates that by manipulating the Content-Encoding header, an attacker can create

desynchronization between how the WAF sees a request and how the Backend processes it. The coreissue is logic asymmetry: the same HTTP protocol, the same set of headers, but two systems (WAF andBackend) process them according to two different logics. When two systems operate independentlywith different assumptions about data processing, the gap between those assumptions is precisely thearea an attacker exploits.

III. Technical Background

3.1 HTTP Content-Encoding

Per RFC 2616 and RFC 7231, the Content-Encoding header indicates what encoding transformations have been applied to the entity-body, primarily for compression to reduce bandwidth. Common values include:

> **gzip / deflate:** Binary compressed data requiring decompression before logical processing. This is the most common value in production environments.
>
> **br (Brotli):** Modern compression algorithm with better performance than gzip, increasingly common in production and supported by most modern browsers.
>
> **identity:** Indicates no encoding has been applied (raw data). This is the default value per RFC when the Content-Encoding header is absent. However, explicitly sending this header is a rare action and is the target of the IBTD exploitation technique.

3.2 Standard WAF Processing Pipeline

A WAF typically operates as a Reverse Proxy between the user and the Backend. The standard processing pipeline includes: receiving HTTP Requests, inspecting the Content-Encoding header to determine the processing pipeline, decompressing the body if needed, performing security inspection, and forwarding safe requests to the Backend. This seemingly straightforward pipeline contains multiple blind spots that attackers can exploit, particularly at the decision point where the processing pipeline is selected based on header values.

Step	Action	Description
1	Receive	WAF receives HTTP Request from client
2	Inspect Header	Check Content-Encoding header to determine processing pipeline
3	Decompress	If compressed, decompress body before security analysis
4	Analyze	Deep Packet Inspection on decompressed/raw body
5	Forward	If safe, forward request to Backend application server

Table 1: Standard WAF Processing Pipeline

IV. Attack Vector 1: ACED

4.1 Definition

Asymmetric Content-Encoding Deception (ACED) exploits differences in error-handling policies between WAF and Backend. The attacker sends a misleading signal about data format to cause a local error on the WAF while remaining acceptable to the Backend. The key to ACED is exploiting the Fail-Open mode - a common configuration designed to ensure high availability, but which creates a severe vulnerability when combined with inconsistent error handling between WAF and Backend.

4.2 Detailed Mechanism

Step 1 - Deceptive Signaling: The attacker crafts an HTTP Request with Content-Encoding: gzip header but a plain-text payload body. The mismatch between header and body is the attack vector. The WAF relies on the header to determine the processing pipeline, while the Backend may process data differently when decompression fails.

Step 2 - WAF Blind Spot (Fail-Open): When the WAF receives the request, it sees the gzip header and routes data to the Decompression Module. Since the actual data is plain text, decompression fails (e.g., "Incorrect header check" in zlib). Here, Logic Asymmetry manifests. To avoid causing service disruption (DoS) when encountering malformed packets, many WAFs are configured with Fail-Open mode - instead of blocking the request, the WAF logs the decompression error and decides to forward the intact packet to the Backend.

Step 3 - Backend Implicit Fallback: The Backend Web Server (IIS, Apache Tomcat, Nginx, Resin) receives the packet and also attempts decompression based on the header, which naturally fails. However, many web servers feature Implicit Fallback or Robustness Mode. When decompression fails, they attempt to read the data as a raw byte stream. Result: the payload is recognized and executed by the web application, completely bypassing WAF protection.

```
ACED Payload: SQL Injection via Gzip Deception
POST /api/v1/users HTTP/1.1
Host: target.example.com
Content-Type: application/x-www-form-urlencoded
Content-Encoding: gzip
Content-Length: 38

id=1' UNION SELECT NULL,NULL,NULL--
```

4.3 ACED Variants

Beyond the basic technique with Content-Encoding: gzip, research has confirmed multiple ACED variants that all operate effectively. Each variant exploits the same Fail-Open principle but through different headers, demonstrating that this vulnerability is systemic and not limited to a specific Content-Encoding value:

Variant	Header Sent	Actual Body	Mechanism
gzip-fake	Content-Encoding: gzip	Plain Text	Fail-Open on gzip decompression error
x-gzip-fake	Content-Encoding: x-gzip	Plain Text	Same as gzip, using x-gzip alias
deflate-fake	Content-Encoding: deflate	Plain Text	Fail-Open on deflate decompression error
br-fake	Content-Encoding: br	Plain Text	Fail-Open on Brotli decompression error
Case Manipulation	Content-Encoding: GZIP / Gzip / GZiP / gZIp	Plain Text	WAF fails to recognize case variations
Quality Value	Content-Encoding: gzip;q=0	Plain Text	WAF mishandles q-value parameter
Double CE	Two Content-Encoding headers	Plain Text	WAF reads only the first header

Table 2: Confirmed ACED Variants

V. Attack Vector 2: IBTD

5.1 Definition

Identity-Based Transparency Deception (IBTD) exploits performance optimization. It leverages the Content-Encoding: identity header to trick the WAF into treating the request as "safe" and "low-cost," thereby bypassing deep inspection. Unlike ACED which relies on error handling, IBTD relies on classification logic - a performance optimization mechanism designed to reduce WAF load but which inadvertently creates a "VIP lane" for malicious payloads.

5.2 Detailed Mechanism

Step 1 - Explicit Default Signaling: The attacker sends a request with Content-Encoding: identity header and a malicious payload body. Explicitly sending this header is a rare action in normal traffic, and this rarity causes some WAFs to handle it in a special way - typically routing it through the optimization branch rather than the full inspection branch.

Step 2 - Shallow Inspection vs. Deep Inspection: To save CPU and reduce latency, WAFs often analyze headers before processing the body. The WAF labels gzip/deflate traffic as "High Cost"

(requiring CPU for decompression) and applies Deep Inspection. Conversely, identity traffic is labeled " Low Cost." Vulnerability: Some WAF configurations assume identity means "clear data with no hidden layers" and apply only Shallow Inspection or skip advanced detection engines entirely.

Step 3 - Transparent Delivery: The payload passes through the WAF like normal traffic, fully RFC-compliant. No errors occur, no exceptions are raised. The WAF forwards the payload to the Backend, which executes it. This "transparency" makes IBTD particularly dangerous: there are no anomalous indicators for monitoring systems or log analysis to detect.

```
IBTD Payload: SQL Injection via Identity Deception
POST /api/v1/login HTTP/1.1
Host: target.example.com
Content-Type: application/x-www-form-urlencoded
Content-Encoding: identity
Content-Length: 52

username=admin&password=' OR 1=1--
```

Traffic Type	Label	Inspection Level	Engines Applied
gzip / deflate	High Cost	Deep Inspection	Signature + Heuristic + Behavioral + ML
No Content-Encoding	Normal	Standard Inspection	Signature + Heuristic
identity (explicit)	Low Cost	Shallow Inspection	Signature only (basic)

Table 3: Inspection Level Comparison by Traffic Type

VI. Comparison and Impact Analysis

Criteria	ACED	IBTD
Attack Vector	Error-Handling Logic	Performance Optimization
Header Exploit	Mismatched (Claims Gzip)	Explicit (Claims Identity)
WAF Reaction	Error -> Fail-Open -> Skip	Classified Low Risk -> Skip Deep
Complexity	Medium	Low
Detectability	Hard (Nearly no logs)	Very Hard (Mimics normal traffic)
Consequence	Complete WAF bypass	Complete WAF bypass
HTTP Response	500 Internal Server Error	200 OK (normal)

Dependency	Fail-Open config	Classification logic

Table 4: Detailed Comparison of ACED and IBTD

6.1 Impact

These techniques allow attackers to perform SQL Injection, RCE, and XXE attacks on systems protected by next-generation WAFs. The ability to exfiltrate data without triggering any alerts (zero-log footprint) makes these techniques particularly dangerous in APT attacks. Furthermore, in the context of data protection regulations such as GDPR, CCPA, and national cybersecurity laws, the failure to detect attacks through WAFs can lead to serious legal consequences.

VII. Real-World Testing: Weaver Ecology OA

To demonstrate the practical feasibility of ACED and IBTD techniques in a real-world environment, testing was conducted on a Weaver Ecology OA (E-cology 8) system deployed at oa.qdhhc.edu.cn (IP: 222.206.231.100). This system uses a Resin application server with the XFire SOAP framework, fronted by a WAF. As a live production system, the test results carry significantly more practical weight than lab environment results.

7.1 Target Environment

Component	Details
Target System	Weaver Ecology OA (E-cology 8)
Endpoint	oa.qdhhc.edu.cn (222.206.231.100)
Application Server	Resin + XFire SOAP Framework
SOAP Services	/services/DocService, /services/HrmService, /services/WorkflowServiceXml
Internal XFire Port	127.0.0.1:8796 (discovered from WSDL)
Java Version	Java 7u48 (confirmed from stack trace)
Protection Layer	WAF (specific vendor unidentified)

Table 5: Target System Information

7.2 WAF Bypass Results

The research tested 17 different bypass techniques across 3 SOAP endpoints, totaling 40 test cases. Result: **100% bypass success rate (40/40)**, with zero requests blocked by the WAF. This demonstrates that logic asymmetry vulnerabilities are not merely theoretical but represent a severe real-world threat to production systems.

Technique	Type	Endpoint	Status	Bypass
gzip-fake	ACED	DocService	500	100%
gzip-fake	ACED	HrmService	500	100%
gzip-fake	ACED	WorkflowServiceXml	500	100%
gzip-identity	IBTD	DocService	500	100%
gzip-identity	IBTD	HrmService	500	100%
gzip-identity	IBTD	WorkflowServiceXml	500	100%
x-gzip-fake	ACED	All 3 endpoints	500	100%
deflate-fake	ACED	All 3 endpoints	500	100%
br-fake	ACED	All 3 endpoints	500	100%
chunked-gzip-fake	ACED +	All 3 endpoints	400	100%
double-ce-raw-socket	ACED +	All 3 endpoints	500	100%
gzip-then-identity	ACED +	All 3 endpoints	500	100%
ce-GZIP/Gzip/GZiP/gZIp	ACED +	Doc/Hrm	500	100%
ce-gzip;q=0 / q=0.0	ACED +	Doc/Hrm	500	100%
ce-gzip, *;q=0	ACED +	Doc/Hrm	500	100%
ce-identity;q=1, gzip;q=0	IBTD+	Doc/Hrm	500	100%

Table 6: Real-World WAF Bypass Results on Weaver Ecology OA (40/40 = 100%)

7.3 Detailed Logs from Real-World Testing

Test Case: gzip-fake on DocService

A request was sent with Content-Encoding: gzip header but raw XML body. The WAF encountered a decompression error and applied Fail-Open policy. The Resin/XFire backend ignored the Content-Encoding header and processed the XML directly, returning a SOAP fault response containing operation error information - confirming the request passed through the WAF and was processed by the backend.

```
ACED Request: gzip-fake on DocService
# Request: ACED - gzip-fake
POST /services/DocService HTTP/1.1
Host: oa.qdhhc.edu.cn
Content-Type: text/xml; charset=UTF-8
Content-Encoding: gzip

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getAllDoc/>
```

```
ACED Response: Successful WAF Bypass - Backend Returns SOAP Fault
# Response: HTTP 500 - SOAP Fault (Backend processed!)
HTTP/1.1 500 Internal Server Error
Content-Type: text/xml; charset=UTF-8
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1
Set-Cookie: ecology_JSessionId=abcLMCAKfXrdHfkQ0fA6z; path=/

<soap:Envelope xmlns:soap="...">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Invalid operation:
        {http://localhost/services/DocService}getAllDoc
      </faultstring>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

Analysis: Although the backend returned HTTP 500 with a SOAP Fault (due to an incorrect operation name), the critical finding is that the request **successfully passed through the WAF** and was processed by the backend. The SOAP Fault "Invalid operation" confirms that Resin/XFire received and parsed the XML, identified the operation name, and only failed at the operation matching step - it was not blocked by the WAF. If the WAF were functioning correctly, the request would have been blocked at the signature inspection stage or rejected due to an invalid body (not valid gzip).

7.4 SOAP Data Extraction

After confirming successful WAF bypass, the research proceeded to extract data from SOAP services. By using correct operation names and valid XML structures (still sent via spoofed Content-Encoding: gzip), sensitive data was successfully exfiltrated:

SOAP Service	Data Extracted	Result
WorkflowServiceXml	661 workflow requests, 53 detailed (22KB-166KB each)	Success
HRM Service	Department list, job titles, employee data	Success (IP restricted)
Doc Service	Document count	Success (0 documents)

Table 7: SOAP Data Extraction Results

7.5 PII Data Exposed

From 53 detailed workflow requests, research discovered a significant amount of Personally Identifiable Information (PII) exposed due to WAF bypass. This data includes phone numbers, national ID numbers, bank account numbers, employee names and IDs, database table names, and 221 sensitive field names. Workflow types containing sensitive data include: Disciplinary decisions, Student affairs stamp requests, Introduction letter approvals, Official vehicle approvals, and Document circulation approvals. This result demonstrates that the real-world consequences of WAF bypass extend far beyond theoretical attack capabilities to large-scale sensitive data exposure.

Data Type	Count	Sensitivity
Unique phone numbers	21	High
National ID numbers	9	Very High
Bank card / account numbers	13	Very High
Employees (name + ID)	35	Medium
Database table names	20+	High
Sensitive field names	221	High

Table 8: Exposed PII Data Statistics

VIII. Remediation

To comprehensively prevent asymmetric attack techniques like ACED and IBTD, organizations must implement the following changes to their security configurations. Each measure is ranked by priority and expected effectiveness, based on root cause analysis of each attack vector.

8.1 For ACED (Consistency-Based Defense)

Fail-Close Configuration: Change the WAF/IPS policy from Fail-Open to Fail-Close when encountering decompression errors. If a packet cannot be decompressed according to its declared format, the system must Drop the packet immediately rather than allowing it through. This is the most critical and effective measure against ACED, as it eliminates the Fail-Open mechanism that the technique relies upon. However, transitioning to Fail-Close may impact service availability for legitimate requests that are corrupted during transmission, so careful deployment with comprehensive monitoring is required.

Header Validity Check (Magic Bytes Validation): The WAF must perform a quick Sanity Check to verify that data actually begins with the "Magic Bytes" of the declared compression format (e.g., 1f 8b for gzip) before processing. If the header declares gzip but the body does not begin with the corresponding magic bytes, this is a clear sign of impersonation and must be blocked immediately. This check should be deployed at the Gateway layer, before the request reaches the decompression module. Real-world testing on Weaver Ecology OA demonstrated that the gzip-fake technique bypassed the WAF easily because the WAF did not perform this validation step.

Backend Synchronization: Configure Web Servers (Tomcat, IIS, Nginx, Resin...) to reject requests with compression headers but invalid actual content, rather than automatically falling back to raw data reading (Implicit Fallback). This measure closes the vulnerability at the Backend side, ensuring that even if the WAF fails to block the request, the Backend will not execute the payload. Specifically, configure Nginx gunzip directive for invalid gzip requests, configure Tomcat to reject requests with encoding errors, and disable IIS/Resin dynamic compression fallback.

8.2 For IBTD (Defense-in-Depth)

Mandatory Deep Packet Inspection (DPI): Remove optimization exceptions for the Content-Encoding: identity header. Every packet, regardless of compression status, must pass through the full signature filters and Heuristic analysis. Security inspection must never be skipped simply because data is uncompressed. Audit the entire WAF traffic classification logic to ensure no branch bypasses essential security inspection engines. Real-world testing showed that gzip-identity bypassed the WAF easily because identity was classified as "low risk."

Header Restriction (Whitelist): Only allow specific Content-Encoding values that the system actually requires. If the application does not require compressed data transmission from clients, disable processing of these headers entirely at the Gateway layer. Specifically, if the application only uses gzip from clients, create a whitelist allowing only the "gzip" value and reject all other values including "

identity." This completely eliminates IBTD exploitation capability.

Zero Trust Architecture: Deploy multi-layer security inspection. If the WAF is bypassed, the Host-based IPS/WAF or application-layer Input Validation must serve as the final checkpoint to identify payloads like SQLi or RCE. Every request must be verified regardless of origin, with no default trust. This represents the transition from a perimeter-based security model to defense-in-depth.

Measure	Target	Priority	Effectiveness
Fail-Close Policy	ACED	Critical	95%+
Sanity Check Magic Bytes	ACED	High	90%+
Backend Synchronization	ACED	High	90%+
Mandatory DPI for identity	IBTD	Critical	95%+
Header Whitelist	IBTD	High	90%+
Zero Trust Architecture	Both	High	Long-term

Table 9: Remediation Priority and Effectiveness Comparison

IX. Conclusion

Both ACED and IBTD demonstrate that over-prioritizing performance and availability can create severe logic vulnerabilities. System operators must review WAF configurations to ensure that security is not compromised by protocol-layer optimizations. Real-world testing on the Weaver Ecology OA system with a 40/40 bypass success rate (100%) and the extraction of PII data belonging to dozens of individuals provides clear evidence that this vulnerability is not merely theoretical but is actively exploitable in practice.

This is not a code bug fixable with a single patch; it is an architectural design flaw demanding a shift in security mindset: from "trusting standards" to "verifying everything." Organizations must recognize that RFC compliance by individual components (WAF, Backend) operating independently is insufficient; what matters is consistency in how protocols are interpreted and processed across all components in the request handling chain.

Furthermore, this research raises a broader question about the perimeter-based security model in general. If WAFs - the primary defense layer for web applications - can be bypassed by exploiting differences in protocol processing logic, then we need to rethink our overall security architecture. Zero Trust Architecture, where every request is verified regardless of origin, may be the necessary direction to counter logic asymmetry-based attacks in the future. The 100% bypass rate on a live production system underscores the urgency of this paradigm shift.